Codex的使用（全query loop）

· 我补了一下上游入口。现在链路已经闭合：QueryEngine.submitMessage() 组装 QueryParams，调用 query()，再进入 queryLoop()；queryLoop() 内部又会调 callModel、

StreamingToolExecutor/runTools、handleStopHooks，最后决定结束还是继续下一轮。下面给你的是基于这些真实文件的源码级讲解。

· 下面这份讲解是按你仓库里的实际文件来的，核心参考：

- /E:/claude-code-sourcemap/CLAUDE.md
- /E:/claude-code-sourcemap/.agents/skills/source_explainer_narrative_control.md
- /E:/claude-code-sourcemap/restored-src/src/QueryEngine.ts
- /E:/claude-code-sourcemap/restored-src/src/query.ts
- /E:/claude-code-sourcemap/restored-src/src/services/tools/StreamingToolExecutor.ts
- /E:/claude-code-sourcemap/restored-src/src/services/tools/toolOrchestration.ts
- /E:/claude-code-sourcemap/restored-src/src/query/stopHooks.ts
- /E:/claude-code-sourcemap/restored-src/src/query/config.ts

先给结论：这里真正的“queryloop”函数名就是 queryLoop，定义在 /E:/claude-code-sourcemap/restored-src/src/query.ts:241。它不是一个简单的“发一次请求拿一次回

复”的循环，而是一个“模型采样 → 收集中间 assistant block → 执行 tool → 把 tool_result 和附件拼回上下文 → 再次采样”的状态机式回环。它的本质是 Claude Code 的

主编排器。

1. 先看完整调用链

完整链路从上游到下游是这样的：

用户输入

→ QueryEngine.submitMessage()

→ processUserInput() 处理 slash command / 附件 / prompt

→ query()

→ queryLoop()

→ 预处理消息、预算、compact/snip/collapse

→ deps.callModel() 流式请求模型

→ 收集 assistant message / tool_use

→ StreamingToolExecutor 或 runTools 执行工具

→ 生成 tool_result / attachment

→ 拼成下一轮 messages

→ 继续 while(true)

直到没有 tool_use、被 stop hook 阻止、超限、报错或中断

上游入口在 /E:/claude-code-sourcemap/restored-src/src/QueryEngine.ts:675：

```
for await (const message of query({
messages,
systemPrompt,
userContext,
systemContext,
canUseTool: wrappedCanUseTool,
toolUseContext: processUserInputContext,
fallbackModel,
querySource: 'sdk',
maxTurns,
taskBudget,
})) {
```

这说明：

- QueryEngine 负责“会话级状态”和“用户输入处理”
- query() / queryLoop() 负责“单次 turn 内的 agentic loop”
- 所有 queryloop 需要的信息，都是这里组装成 QueryParams 传进去的

2. query() 只是一个薄包装

/E:/claude-code-sourcemap/restored-src/src/query.ts:219 的 query() 很薄：

```
export async function* query(params: QueryParams) {
const consumedCommandUuids: string[] = []
const terminal = yield* queryLoop(params, consumedCommandUuids)
for (const uuid of consumedCommandUuids) {
 notifyCommandLifecycle(uuid, 'completed')
}
return terminal
}
```

它只做两件事：

- 把真正工作委托给 queryLoop
- 在 queryLoop 正常结束后，补发队列命令的 completed 生命周期通知

所以真正值得分析的是 queryLoop()。

3. queryLoop() 的总体结构

queryLoop 定义在 /E:/claude-code-sourcemap/restored-src/src/query.ts:241。

它的结构可以抽成：

```
初始化不可变参数
初始化 state
初始化预算/配置/预取器

while (true) {
从 state 解包本轮变量
```

```
构造 queryTracking
裁剪/压缩/整理 messagesForQuery
选择模型，建立 streaming tool executor
调用模型流
收集 assistantMessages / toolUseBlocks / toolResults

if 被中断 -> 结束
if 没有 tool_use -> 走 stop hooks / 错误恢复 / 结束或续轮
else -> 执行工具，生成 tool_result / attachment
    拼出下一轮 state
    continue
}
```

你可以把它理解成一个显式状态机，而不是递归函数。虽然逻辑上是“多轮”，但实现上是 while(true) + state。

4. 入口参数和 state 是什么

QueryParams 在 /E:/claude-code-sourcemap/restored-src/src/query.ts:185 定义，重要字段有：

- messages: 当前对话历史
- systemPrompt
- userContext / systemContext
- canUseTool
- toolUseContext
- fallbackModel
- querySource
- maxOutputTokensOverride
- maxTurns
- taskBudget
- deps

State 在 /E:/claude-code-sourcemap/restored-src/src/query.ts:203 定义。它是 queryloop 跨迭代携带的核心状态：

- messages: 下一轮要带上的消息数组
- toolUseContext: 工具上下文
- autoCompactTracking: autocompact 的追踪状态
- maxOutputTokensRecoveryCount: OTK 恢复计数
- hasAttemptedReactiveCompact: 是否已经做过 reactive compact
- maxOutputTokensOverride: 输出 token 上限覆盖
- pendingToolUseSummary: 上一轮工具摘要 promise
- stopHookActive: stop hook 是否已激活
- turnCount: 当前是第几轮
- transition: 上一轮为什么继续

这是整个函数最关键的设计点之一：它避免了大量 scattered mutable variables，而是集中到一个 state，每次 continue 时整体重建。

5. 逐段解析 queryLoop()

下面按代码块解析。

5.1 初始化区

/E:/claude-code-sourcemap/restored-src/src/query.ts:253

这里做了几件事：

1. 解构不变参数：

const {

systemPrompt,

userContext,

systemContext,

```
canUseTool,
fallbackModel,
querySource,
maxTurns,
skipCacheWrite,
} = params
```

这些值在整个 loop 生命周期内不变。

2. 解析依赖注入：

```
const deps = params.deps ?? productionDeps()
```

这是为了测试和生产隔离。queryLoop 不直接绑死外部实现。

3. 初始化 state：

```
let state: State = {
messages: params.messages,
toolUseContext: params.toolUseContext,
maxOutputTokensOverride: params.maxOutputTokensOverride,
autoCompactTracking: undefined,
stopHookActive: undefined,
maxOutputTokensRecoveryCount: 0,
hasAttemptedReactiveCompact: false,
turnCount: 1,
pendingToolUseSummary: undefined,
transition: undefined,
}
```

初始轮次是 1。

4. 初始化 token budget tracker 和 taskBudgetRemaining

   const budgetTracker = feature('TOKEN_BUDGET') ? createBudgetTracker() : null

   let taskBudgetRemaining: number | undefined = undefined

   taskBudgetRemaining 很重要：它是跨 compaction 的剩余上下文预算校正值。

5. snapshot config：

   const config = buildQueryConfig()

   对应 /E:/claude-code-sourcemap/restored-src/src/query/config.ts:29，把一些 env/statsig gate 固化到这次 query 的生命周期里，比如：

- streamingToolExecution
- emitToolUseSummaries
- isAnt
- fastModeEnabled

6. 启动 memory prefetch：

   using pendingMemoryPrefetch = startRelevantMemoryPrefetch(

   state.messages,

   state.toolUseContext,

)

   这是个并发优化：先后台找记忆，后面合适时机再消费。

   5.2 while(true) 每轮开始

   /E:/claude-code-sourcemap/restored-src/src/query.ts:307

每次迭代都先从 state 解包：

```
let { toolUseContext } = state
const {
messages,
autoCompactTracking,
maxOutputTokensRecoveryCount,
hasAttemptedReactiveCompact,
maxOutputTokensOverride,
pendingToolUseSummary,
stopHookActive,
turnCount,
} = state
```

然后做两件关键事情：

1. 启动 skill prefetch

```
const pendingSkillPrefetch = skillPrefetch?.startSkillDiscoveryPrefetch(
null,
messages,
toolUseContext,
)
```

2. 初始化 queryTracking

```
const queryTracking = toolUseContext.queryTracking
? { chainId: toolUseContext.queryTracking.chainId, depth:
toolUseContext.queryTracking.depth + 1 }
: { chainId: deps.uuid(), depth: 0 }
```

这是“同一用户 turn 内的 query 链”追踪器。第一次 query 深度是 0，后续 tool loop 的 follow-up query 深度递增。

随后把它写回 toolUseContext：

```
toolUseContext = {
...toolUseContext,
queryTracking,
}
```

5.3 生成 messagesForQuery 并做预处理

/E:/claude-code-sourcemap/restored-src/src/query.ts:365

这里是 queryloop 非常重的一段：在真正调模型前先整上下文。

顺序是：

```
getMessagesAfterCompactBoundary
→ applyToolResultBudget
→ HISTORY_SNIP
→ microcompact
→ contextCollapse
→ autocompact
→ 更新 toolUseContext.messages
```

先裁到最后一个 compact boundary 之后：

```
let messagesForQuery = [...getMessagesAfterCompactBoundary(messages)]
```

然后对 tool_result 做内容预算限制：

```
messagesForQuery = await applyToolResultBudget(...)
```

再做 snip：

```
const snipResult = snipModule!.snipCompactIfNeeded(messagesForQuery)
messagesForQuery = snipResult.messages
snipTokensFreed = snipResult.tokensFreed
if (snipResult.boundaryMessage) yield snipResult.boundaryMessage
```

再做 microcompact：

```
const microcompactResult = await deps.microcompact(
messagesForQuery,
toolUseContext,
querySource,
)
messagesForQuery = microcompactResult.messages
```

再做 context collapse：

```
const collapseResult = await contextCollapse.applyCollapsesIfNeeded(...)
messagesForQuery = collapseResult.messages
```

最后 autocompact：

```
const { compactionResult, consecutiveFailures } = await deps.autocompact(...)
```

如果 compact 成功，就：

- 记录 analytics

- 修正 taskBudgetRemaining
- reset compact tracking
- 构造 postCompactMessages
- yield 这些 compact 边界/摘要消息
- 用 postCompactMessages 继续当前查询

这点很关键：compact 并不是“结束这轮，下轮再继续”，而是“在当前 queryloop 内即时改写待查询上下文，然后继续发模型请求”。

最后更新 toolUseContext.messages = messagesForQuery，保证后续工具执行和 hook 看见的是压缩后的真实 query 输入。

5.4 准备本轮采样

/E:/claude-code-sourcemap/restored-src/src/query.ts:551

这里初始化采样阶段的暂存容器：

```
const assistantMessages: AssistantMessage[] = []
const toolResults: (UserMessage | AttachmentMessage)[] = []
const toolUseBlocks: ToolUseBlock[] = []
let needsFollowUp = false
```

语义分别是：

- assistantMessages: 本轮模型流式产生的 assistant block 们
- toolResults: 本轮工具执行产生的 user/attachment 消息
- toolUseBlocks: 本轮 assistant 中抽出的 tool_use blocks
- needsFollowUp: 本轮是否需要继续下一轮 query

然后决定是否启用 StreamingToolExecutor：

```
const useStreamingToolExecution = config.gates.streamingToolExecution
let streamingToolExecutor = useStreamingToolExecution
? new StreamingToolExecutor(...)
: null
```

再根据 permission mode、上下文长度等选择 runtime model：

```
let currentModel = getRuntimeMainLoopModel({...})
```

5.5 调模型前的 blocking limit 防线

/E:/claude-code-sourcemap/restored-src/src/query.ts:592

如果 auto-compact 没开或当前场景不适合自动恢复，就先按 token 数硬拦截：

```
const { isAtBlockingLimit } = calculateTokenWarningState(
tokenCountWithEstimation(messagesForQuery) - snipTokensFreed,
toolUseContext.options.mainLoopModel,
)
if (isAtBlockingLimit) {
yield createAssistantAPIErrorMessage({
  content: PROMPT_TOO_LONG_ERROR_MESSAGE,
  error: 'invalid_request',
})
return { reason: 'blocking_limit' }
}
```

也就是：如果 prompt 太长，连 API 都不打，直接报错返回。

5.6 核心：流式模型调用

/E:/claude-code-sourcemap/restored-src/src/query.ts:649

这一段是 queryloop 的心脏。

外层先包一个 attemptWithFallback，支持模型 fallback 重试：

```
let attemptWithFallback = true
while (attemptWithFallback) {
attemptWithFallback = false
try {
  for await (const message of deps.callModel({...})) {
    ...
  }
} catch (innerError) {
  if (innerError instanceof FallbackTriggeredError && fallbackModel) {
    ...
    continue
  }
  throw innerError
}
}
```

真正模型流来自 deps.callModel()。这会持续产出：

- stream_event
- assistant 消息块
- 错误 assistant
- 其他控制消息

在每个流式 message 上，queryloop 做了几件非常细的事。

第一，处理 streaming fallback 的残留清理：

```
if (streamingFallbackOccured) {
for (const msg of assistantMessages) {
  yield { type: 'tombstone', message: msg }
}
assistantMessages.length = 0
toolResults.length = 0
toolUseBlocks.length = 0
needsFollowUp = false
streamingToolExecutor.discard()
streamingToolExecutor = new StreamingToolExecutor(...)
}
```

意思是：如果这次流式请求中途 fallback 了，之前已经吐给 UI 的 assistant partial messages 都要 tombstone 掉，否则 transcript 和 thinking signature 会坏。

第二，对 tool_use 做 backfill observable input，但只改 yield 给外部的副本，不改原消息：

```
let yieldMessage: typeof message = message
if (message.type === 'assistant') {
...
if (tool?.backfillObservableInput) {
  ...
  yieldMessage = clonedMessage
}
}
```

这是为了让 UI/SDK 看到扩展字段，但保留送回 API 的原始字节，避免 prompt caching hash mismatch。

第三，决定某些错误消息是否“先压住不 yield”：

```
let withheld = false
if (contextCollapse?.isWithheldPromptTooLong(...)) withheld = true
if (reactiveCompact?.isWithheldPromptTooLong(message)) withheld = true
if (mediaRecoveryEnabled && reactiveCompact?.isWithheldMediaSizeError(message)) withheld = true
if (isWithheldMaxOutputTokens(message)) withheld = true
if (!withheld) yield yieldMessage
```

这个设计很关键。因为有些错误并不一定要立刻暴露给用户：

- prompt too long 也许可以 reactive compact 后重试
- media size 错误也许能剥离附件后重试
- max_output_tokens 也许可以自动续写/升 token cap

所以先藏起来，等后面恢复逻辑判断是否能救。

第四，收集 assistant / tool_use：

```
assistantMessages.push(message)
const msgToolUseBlocks = message.message.content.filter(...)
if (msgToolUseBlocks.length > 0) {
toolUseBlocks.push(...msgToolUseBlocks)
needsFollowUp = true
}
```

needsFollowUp 的语义是：只要看到了 tool_use，这轮就还没结束。

第五，如果启用了 streaming tool execution，就在 tool_use 一出现时立刻排队执行：

```
for (const toolBlock of msgToolUseBlocks) {
streamingToolExecutor.addTool(toolBlock, message)
```

}

第六，在模型继续 streaming 的同时，轮询已完成工具结果并立刻 yield：

```
for (const result of streamingToolExecutor.getCompletedResults()) {
if (result.message) {
  yield result.message
  toolResults.push(...normalizeMessagesForAPI([result.message], ...).filter(_ => _.type ===
'user'))
}
}
```

这意味着这里的架构不是“等模型全部结束再统一跑工具”，而是支持某种程度的 assistant streaming 与工具执行并行。

5.7 fallback / 异常处理

/E:/claude-code-sourcemap/restored-src/src/query.ts:892

如果模型触发 fallback：

- 给前一轮未完成 tool_use 补 synthetic tool_result
- 清空本轮 assistant/toolResults/toolUseBlocks
- 重建 executor
- 切到 fallbackModel
- 必要时 strip thinking signatures
- yield 一条 warning system message
- continue 重新打这一轮

如果整个模型调用抛错：

- 记录 analytics

- 对图片大小/resize错误给用户友好提示
- 用 yieldMissingToolResultBlocks() 给所有未完成 tool_use 补错
- 再 yield 一个 assistant API error
- 返回 { reason: 'model_error' }

yieldMissingToolResultBlocks() 在 /E:/claude-code-sourcemap/restored-src/src/query.ts:116 定义，本质是“别让 tool_use 悬空”。

5.8 流式结束后，先处理中断

/E:/claude-code-sourcemap/restored-src/src/query.ts:998

如果用户在 streaming 中途 abort：

- 若用了 StreamingToolExecutor，要把剩余结果 drain 完，生成 synthetic tool_result
- 否则直接给 assistantMessages 里的每个 tool_use 补 Interrupted by user
- 做 computer use cleanup
- 视情况 yield interruption message
- return { reason: 'aborted_streaming' }

这一步必须放在最前，否则会留下 orphan tool_use。

5.9 如果本轮没有 tool_use：进入“收尾/恢复/结束”路径

/E:/claude-code-sourcemap/restored-src/src/query.ts:1052

这里是“模型已经回答完，不需要跑工具了，那这轮是不是就结束”的判断区。

先把上一轮异步生成的工具摘要吐出来：

```
if (pendingToolUseSummary) {
const summary = await pendingToolUseSummary
if (summary) yield summary
```

}

然后看 needsFollowUp。如果是 false，说明这轮 assistant 最终没有 tool_use，要么结束，要么做恢复重试。

恢复逻辑分三类：

1. prompt-too-long / media-size recover
2. max_output_tokens recover
3. stop hooks / token budget continuation

A. prompt-too-long / media-size recover

/E:/claude-code-sourcemap/restored-src/src/query.ts:1063

先判断最后一条 assistant 是不是被 withheld 的 413 或媒体尺寸错误。

如果是 withheld 413，先试 context collapse drain：

```
const drained = contextCollapse.recoverFromOverflow(messagesForQuery, querySource)
if (drained.committed > 0) {
state = { ... transition: { reason: 'collapse_drain_retry' } }
continue
}
```

如果还不行，再试 reactive compact：

```
const compacted = await reactiveCompact.tryReactiveCompact(...)
if (compacted) {
yield postCompactMessages
state = { messages: postCompactMessages, hasAttemptedReactiveCompact: true, ... }
continue
```

}

如果也救不了，就把 withheld 错误真正吐给用户，然后返回。

B. max_output_tokens recover

/E:/claude-code-sourcemap/restored-src/src/query.ts:1183

这是另一个非常典型的 queryloop 恢复路径。

先看是不是可以从默认 8k cap 升到 ESCALATED_MAX_TOKENS：

if (capEnabled && maxOutputTokensOverride === undefined && !process.env.CLAUDE_CODE_MAX_OUTPUT_TOKENS) {

state = { ..., maxOutputTokensOverride: ESCALATED_MAX_TOKENS, transition: { reason: 'max_output_tokens_escalate' } }

continue

}

如果已经升过 cap 或不能升，就走 meta user message 续写：

const recoveryMessage = createUserMessage({

content: `Output token limit hit. Resume directly ...`,

isMeta: true,

})

state = {

messages: [...messagesForQuery, ...assistantMessages, recoveryMessage],

maxOutputTokensRecoveryCount: maxOutputTokensRecoveryCount + 1,

transition: { reason: 'max_output_tokens_recovery', attempt: ... },

}

continue

最多恢复 MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3 次。超过就把 withheld 错误吐出来。

C. stop hooks

/E:/claude-code-sourcemap/restored-src/src/query.ts:1256

如果最后是 API error message，就不跑 stop hooks 了，直接返回 completed。

否则调用：

const stopHookResult = yield* handleStopHooks(...)

handleStopHooks() 在 /E:/claude-code-sourcemap/restored-src/src/query/stopHooks.ts:65。

它会：

- 执行 stop hooks
- yield progress / attachment / summary
- 收集 blockingErrors
- 判断是否 preventContinuation

若 preventContinuation，queryloop 直接返回。

若有 blockingErrors，就把这些错误变成 meta user message 拼回上下文，再 continue 下一轮，让模型依据 hook 错误继续修正。

D. token budget continuation

/E:/claude-code-sourcemap/restored-src/src/query.ts:1304

如果 feature 打开，会检查这轮是否应该为了 token budget 再多跑一轮：

const decision = checkTokenBudget(...)

```
if (decision.action === 'continue') {
state = {
  messages: [...messagesForQuery, ...assistantMessages, createUserMessage({ content: decision.nudgeMessage, isMeta: true })],
  transition: { reason: 'token_budget_continuation' },
}
continue
}
```

否则记录 completion event，然后 return { reason: 'completed' }。

这一大块说明：没有 tool_use 并不意味着立即结束。queryloop 还会插入多种“自动继续”策略。

5.10 如果本轮有 tool_use：进入工具执行路径

/E:/claude-code-sourcemap/restored-src/src/query.ts:1356

这块是第二个心脏区。

先决定工具执行器来源：

```
const toolUpdates = streamingToolExecutor
? streamingToolExecutor.getRemainingResults()
: runTools(toolUseBlocks, assistantMessages, canUseTool, toolUseContext)
```

如果 streaming executor 开着，那么模型 streaming 阶段其实已经开始执行一部分工具了，这里只是把剩余结果 drain 完。

如果没开，就走传统 runTools()。

然后消费所有 update：

```
for await (const update of toolUpdates) {
if (update.message) {
  yield update.message
  ...
  toolResults.push(...normalizeMessagesForAPI([update.message], ...).filter(_ => _.type ===
'user'))
}
if (update.newContext) {
  updatedToolUseContext = { ...update.newContext, queryTracking }
}
}
```

这一步非常关键：

- 工具执行会产出 message
- 工具执行还可能修改 ToolUseContext
- queryloop 会把这些修改过的 context 带到下一轮

这说明工具不只是“纯函数返回字符串”，而是能影响后续调度环境。

5.11 StreamingToolExecutor 是怎么工作的

看 /E:/claude-code-sourcemap/restored-src/src/services/tools/StreamingToolExecutor.ts。

它维护一个 tools: TrackedTool[] 队列。每个 tool 有：

- status
- isConcurrencySafe
- results
- pendingProgress
- contextModifiers

addTool() 时就会判断工具是否 concurrency-safe：

```
const isConcurrencySafe = parsedInput?.success
? Boolean(toolDefinition.isConcurrencySafe(parsedInput.data))
: false
```

然后入队并 processQueue()。

processQueue() 的规则是：

- 如果当前执行中的工具都可并发，新的并发安全工具可以一起跑
- 非并发安全工具必须独占
- 为了顺序稳定，遇到不能执行的非并发安全工具就 stop

这是 Claude Code 并发工具调度的核心保守策略。

executeTool() 里又做了很多安全处理：

- 若已 abort / discarded / sibling error，直接生成 synthetic error message
- 为每个工具创建 child abort controller
- 允许 sibling bash error 传播，杀掉同批其他工具
- 收集 progress、结果和 contextModifier

5.12 传统 runTools() 是怎么工作的

看 /E:/claude-code-sourcemap/restored-src/src/services/tools/toolOrchestration.ts:19。

它先 partitionToolCalls()：

- 并发安全工具按连续片段聚成 batch
- 非并发安全工具单独成 batch

然后：

- 并发 batch 用 runToolsConcurrently()
- 非并发 batch 用 runToolsSerially()

并发版会延迟应用 contextModifier，等整个 batch 完成后按原始 block 顺序回放 modifier，避免并发写 context 乱序。

这是个很细但很对的设计。

5.13 工具执行后，插入摘要、附件、内存、队列消息

/E:/claude-code-sourcemap/restored-src/src/query.ts:1407

先异步生成下一轮可显示的工具摘要：

nextPendingToolUseSummary = generateToolUseSummary(...).then(...).catch(() => null)

注意它不是本轮立即 await，而是存在 state.pendingToolUseSummary 里，等下一轮没 tool_use 时再吐出来。这是典型的 latency hiding。

然后处理工具执行中断，如果 abort 就返回。

再往 toolResults 里追加各种 attachment：

- queued command attachments
- memory prefetch attachments
- skill discovery attachments

这些 attachment 都会被当作下一轮的上下文一部分。

这里尤其要注意一条注释：

// the API will error if we interleave tool_result messages with regular user messages.

所以 attachment 注入被放在工具执行完成之后。

5.14 构造下一轮 state，形成真正的 loop

/E:/claude-code-sourcemap/restored-src/src/query.ts:1669

最后拼下一轮：

```
const next: State = {
messages: [...messagesForQuery, ...assistantMessages, ...toolResults],
toolUseContext: toolUseContextWithQueryTracking,
autoCompactTracking: tracking,
turnCount: nextTurnCount,
maxOutputTokensRecoveryCount: 0,
hasAttemptedReactiveCompact: false,
pendingToolUseSummary: nextPendingToolUseSummary,
maxOutputTokensOverride: undefined,
stopHookActive,
transition: { reason: 'next_turn' },
}
state = next
```

这就是 queryloop 的闭环点。

它把本轮输入、assistant输出、tool结果，全部串成下一轮的 messages，然后回到 while(true) 顶部继续。

所以完整闭环是：

messagesForQuery

→ 模型输出 assistantMessages

→ 从 assistantMessages 抽 tool_use

→ 工具执行得到 toolResults

→ next.messages = oldQuery + assistantMessages + toolResults

→ 下一轮模型看到的是包含 tool_result 的新上下文

这就是 Claude-style agent loop 的核心。

6. 从架构上看，这个 queryloop 为什么这样设计

结合你的 /E:/claude-code-sourcemap/CLAUDE.md 和 skill 文件强调的“源码事实优先、流程可追溯”，这个 queryLoop 的设计意图很清楚：

2. 它把“消息历史”作为唯一主数据，把所有恢复、compact、hook、tool-result 都编码成消息或 state 变迁。
3. 它把“流式模型调用”和“工具执行”部分并行化，以缩短端到端延迟。
4. 它把各种失败恢复都做成“同一 loop 内 continue”，而不是把错误扔给上层重新组织。
5. 它允许工具通过 newContext/contextModifier 改变后续运行环境，说明工具是编排的一等公民，不只是 side effect。

7. 你可以把 queryloop 记成这 8 步

如果你想脑内建立一个稳定模型，我建议记下面这个版本：

3. 流式请求模型，持续收集 assistant block。
4. 一旦看见 tool_use，就记入 toolUseBlocks，必要时立刻并发跑工具。
5. 流结束后，如果没有 tool_use，先做错误恢复和 stop hooks，再决定结束还是继续。
6. 如果有 tool_use，执行剩余工具，产出 tool_result 和 attachment。
7. 把 messagesForQuery + assistantMessages + toolResults 拼成下一轮 state.messages。

- query() 外壳：/E:/claude-code-sourcemap/restored-src/src/query.ts:219
- queryLoop() 定义和 state 初始化：/E:/claude-code-sourcemap/restored-src/src/query.ts:241
- 上下文预处理链：/E:/claude-code-sourcemap/restored-src/src/query.ts:365
- 模型 streaming 主循环：/E:/claude-code-sourcemap/restored-src/src/query.ts:658

- withheld/recovery 逻辑：/E:/claude-code-sourcemap/restored-src/src/query.ts:798
- 无 tool_use 时的 stop/recover/complete 路径：/E:/claude-code-sourcemap/restored-src/src/query.ts:1060
- 有 tool_use 时的 tool execution 路径：/E:/claude-code-sourcemap/restored-src/src/query.ts:1356
- 下一轮 state 拼装：/E:/claude-code-sourcemap/restored-src/src/query.ts:1710
- 上游入口 QueryEngine.submitMessage()：/E:/claude-code-sourcemap/restored-src/src/QueryEngine.ts:675

如果你愿意，我下一条可以继续做两种更深的版本之一：

1. “逐行精讲 queryLoop 241-1724”，我按行号几乎一段不漏地解释。
2. “只深挖一个支线”，比如 max_output_tokens 恢复链、streaming tool execution、stop hooks、compact 链。